Table des matières

Cours 05 — Les codes 1

Cours 05 — Les codes

Cours 5/12 · La Brûlerie — Formation Data Engineering

Sources	Ingestion	RAW
d'où vient la donnée	aller la chercher, et la charger	la déposer, la ranger, la traiter
✓ ici · Tenir le pipeline, ce qui traverse les trois étapes : le code, l'orchestration, la qualité	✓	✓

À ouvrir après avoir essayé. Le code garde les commentaires numérotés du cours, pour comparer ligne par ligne. Le `.gitignore` se montre en entier, dans son nouvel état.

01_de/pipeline.py

```python
# 01_de/pipeline.py : la porte d'entrée du pipeline, qui enchaîne les étapes et rend un code de sortie
import sys
import duckdb
from charger import charger

def bronze(jusqua):
    # 1. le bronze, refait en entier ; son contrôle doit compter autant de tickets des deux côtés
    source, tickets = duckdb.sql(open("01_de/sql/bronze.sql", encoding="utf-8").read()).fetchone()
    if source != tickets:
        raise ValueError(f"{source} tickets à la source, {tickets} dans le bronze")
    return f"{tickets} tickets rangés"

def raw(jusqua):
    # 2. la RAW, chargée de ce qui est arrivé jusqu'au jour donné
    with duckdb.connect("plateforme/entrepot/brulerie.duckdb") as con:
        charger(con, jusqua)
        return f"{con.execute('SELECT COUNT(*) FROM journal').fetchone()[0]} fichiers au journal"

def lancer(jusqua):
    # 3. chaque étape tracée ; la première qui échoue arrête tout, et le code de sortie vaut 1
    for nom, etape in (("bronze", bronze), ("raw", raw)):
```


© 2026 iAlexMG. Tous droits réservés.

```
        try:
            print(f"{nom} : {etape(jusqua)}")
        except Exception as erreur:
            print(f"{nom} : échec, {erreur}")
            return 1
    return 0

if __name__ == "__main__":
    sys.exit(lancer(sys.argv[1]))
```

01_de/pytest.ini

```
# 01_de/pytest.ini : où pytest cherche les tests, et le code qu'ils importent
[pytest]
# 1. le dossier du dépôt, pour que « from charger import charger » trouve charger.py
pythonpath = .
testpaths = tests
```

01_de/tests/test_charger.py

```
# 01_de/tests/test_charger.py : relancer le chargement ne double rien
import duckdb

from charger import charger

def test_relancer_ne_double_rien(tmp_path, monkeypatch):
    # 1. un projet jetable : deux exports de tickets, livrés le 1er et le 2 juin
    caisse = tmp_path / "sources" / "caisse" / "2025-06"
    caisse.mkdir(parents=True)
    for jour in ("01", "02"):
        (caisse / f"tickets_2025-06-{jour}_202506{jour}1707.csv").write_text(f"ticket_id\n{jour}\n")
    monkeypatch.chdir(tmp_path)
    con = duckdb.connect()
    # 2. charger le 1er juin : un seul fichier est arrivé
    charger(con, "2025-06-01")
    assert con.execute("SELECT COUNT(*) FROM raw_tickets").fetchone()[0] == 1
    # 3. charger le mois, puis le relancer : deux fichiers au journal, deux tickets, rien de doublé
    charger(con, "2025-06-30")
    charger(con, "2025-06-30")
    assert con.execute("SELECT COUNT(*) FROM journal").fetchone()[0] == 2
    assert con.execute("SELECT COUNT(*) FROM raw_tickets").fetchone()[0] == 2
```


© 2026 iAlexMG. Tous droits réservés.

01_de/.gitignore

```
# Fichiers compilés par Python
__pycache__/
# Le cache de pytest
.pytest_cache/
```


© 2026 iAlexMG. Tous droits réservés.